

```

#include <iostream.h>
#include <conio.h>
class baseA {
public :
virtual void display () {
cout<< "One \n";
}
};
class derivedB : public baseA
{
    public:
    virtual void display(){
        cout<< "Two\n"; }
};
class derivedC: public derivedB
{
    public:
    virtual void display ( ) {
        cout<< "Three \n"; }
};
void main ( ) {
    //define three objects
    baseA obja;
    derivedB objb;
    derivedC objc;
    base A *ptr [3]; //define an array of pointers to baseA
    ptr [0] = &obja;
    ptr [1] = &objb;
    ptr [2] = &objc;
    for ( int i = 0; i <=2; i ++ )
        ptr [i]->display ( ); //same message for all objects
    getch ( );
}

```

Output

One

Two

Three

The program listed below illustrates the static binding of the member functions of a class. In program there are two classes student and academic. The class academic is derived from class student. The two member function getdata and display are defined for both the classes. *obj is defined for class student, the address of which is stored in the object of the class academic. The functions getdata () and display () of student class are invoked by the pointer to the class.

```

#include<iostream.h>
#include<conio.h>
class student {
private:
int rollno;
char name [20];
public:
void getdata ( );
void display ( );

```

```

};
class academic: public student {
private:
char stream;
public:
void getdata ( );
void display ( ) ;
};
void student:: getdata ( )
{
    cout<< "enter rollno\n";
    cin>>rollno;
    cout<< "enter name \n";
    cin>>name;
}
void student:: display ( )
{
    cout<< "the student's roll number is "<<rollno<< "and name is"<<name ;
    cout<< endl;
}
void academic :: getdata ( )
{
    cout<< "enter stream of a student? \n";
    cin >>stream;
}
void academic :: display ( ) {
    cout<< "students stream \n";
    cout <<stream<< endl;
}
void main ( )
{
    student *ptr ;
    academic obj ;
    ptr=&obj;
    ptr->getdata ( ) ;
    ptr->display ( ) ;
    getche ( ) ;
}

```

output
 enter rollno
 25
 enter name
 raghu
 the student's roll number is 25 and name is raghu

The program listed below illustrates the dynamic binding of member functions of a class. In this program there are two classes student and academic. The class academic is derived from student. Student function has two virtual functions getdata () and display (). The pointer for student class is defined and object . for academic class is created. The pointer is assigned the address of the object and function of derived class are invoked by pointer to student.

```

#include <iostream.h>
#include <conio.h>
class student {

```

```

private:
introllno;
char name [20];
public:
virtual void getdata ( );
virtual void display ( );
};
class academic: public student {
private :
char stream[10];
public:
void getdata { };
void display ( ) ;
};
void student: : getdata ( )
{
    cout<< "enter rollno\n";
    cin >> rollno;
    cout<< "enter name \n";
    cin >>name;
}
void student:: display ( )
{
    cout<< "the student's roll number is"<<rollno<< "and name is"<<name;
    cout<< endl;
}
void academic: : getdata ( )
{
    cout << "enter stream of a student? \n";
    cin>> stream;
}
void academic:: display ( )
{
    cout<< "students stream \n";
    cout<< stream << endl;
}
void main ( )
{
    student *ptr ;
    academic obj ;
    ptr = &obj ;
    ptr->getdata ( );
    ptr->dlsplay ( );
    getch ( );
}

```

output
 enter stream of a student?
 Btech
 students stream
 Btech

LECTURE-32

Pure Virtual Functions

Generally a function is declared virtual inside a base class and we redefine it the derived classes. The function declared in the base class seldom performs any task.

The following program demonstrates how a pure virtual function is defined, declared and invoked from the object of a derived class through the pointer of the base class. In the example there are two classes employee and grade. The class employee is base class and the grade is derived class. The functions getdata () and display () are declared for both the classes. For the class employee the functions are defined with empty body or no code inside the function. The code is written for the grade class. The methods of the derived class are invoked by the pointer to the base class.

```
#include<iostream.h>
#include<conio.h>
class employee {
int code
char name [20] ;
public:
virtual void getdata ( ) ;
virtual void display ( ) ;
};
class grade: public employee
{
    char grd [90] ;
    float salary ;
public :
    void getdata ( ) ;
    void display ( );
};
void employee :: getdata ( )
{
}
void employee:: display ( )
{
}
void grade : : getdata ( )
{
    cout<< " enter employee's grade ";
    cin> > grd ;
    cout<< "\n enter the salary " ;
    cin>> salary;
}
void grade : : display ( )
{
    cout<<" Grade salary \n";
    cout<< grd<< " "<< salary<< endl;
```

```

    }
    void main ( )
    {
        employee *ptr ;
        grade obj ;
        ptr = &obj ;
        ptr->getdata ( ) ;
        ptr->display ( ) ;
        getche ( ) ;
    }

```

Output

```

enter employee's grade A
enter the salary 250000
Grade salary
A      250000

```

Object Slicing:

In C++, a derived class object can be assigned to a base class object, but the other way is not possible.

```

class Base { int x, y; };

class Derived : public Base { int z, w; };

int main()
{
    Derived d;
    Base b = d; // Object Slicing, z and w of d are sliced off
}

```

Object Slicing happens when a derived class object is assigned to a base class object, additional attributes of a derived class object are sliced off to form the base class object.

```

#include <iostream>
using namespace std;

class Base
{
protected:
    int i;
public:
    Base(int a)      { i = a; }
    virtual void display()
    { cout << "I am Base class object, i = " << i << endl; }
};

class Derived : public Base
{
    int j;
public:
    Derived(int a, int b) : Base(a) { j = b; }
    virtual void display()
    { cout << "I am Derived class object, i = "
      << i << ", j = " << j << endl; }
};

```

```
// Global method, Base class object is passed by value
void somefunc (Base obj)
{
    obj.display();
}

int main()
{
    Base b(33);
    Derived d(45, 54);
    somefunc(b);
    somefunc(d); // Object Slicing, the member j of d is sliced off
    return 0;
}
```

Output:

```
I am Base class object, i = 33
I am Base class object, i = 45
```

We can avoid above unexpected behavior with the use of pointers or references. Object slicing doesn't occur when pointers or references to objects are passed as function arguments since a pointer or reference of any type takes same amount of memory. For example, if we change the global method myfunc() in the above program to following, object slicing doesn't happen.

```
// rest of code is similar to above
void somefunc (Base &obj)
{
    obj.display();
}
// rest of code is similar to above
```

Output:

```
I am Base class object, i = 33
I am Derived class object, i = 45, j = 54
```

We get the same output if we use pointers and change the program to following.

```
// rest of code is similar to above
void somefunc (Base *objp)
{
    objp->display();
}
int main()
{
    Base *bp = new Base(33) ;
    Derived *dp = new Derived(45, 54);
    somefunc(bp);
    somefunc(dp); // No Object Slicing
    return 0;
}
```

Output:

```
I am Base class object, i = 33
I am Derived class object, i = 45, j = 54
```

Object slicing can be prevented by making the base class function pure virtual there by disallowing object creation. It is not possible to create the object of a class which contains a pure virtual method.

LECTURE-33

C++ Function Overriding

If base class and derived class have member functions with same name and arguments. If you create an object of derived class and write code to access that member function then, the member function in derived class is only invoked, i.e., the member function of derived class overrides the member function of base class. This feature in C++ programming is known as function overriding.

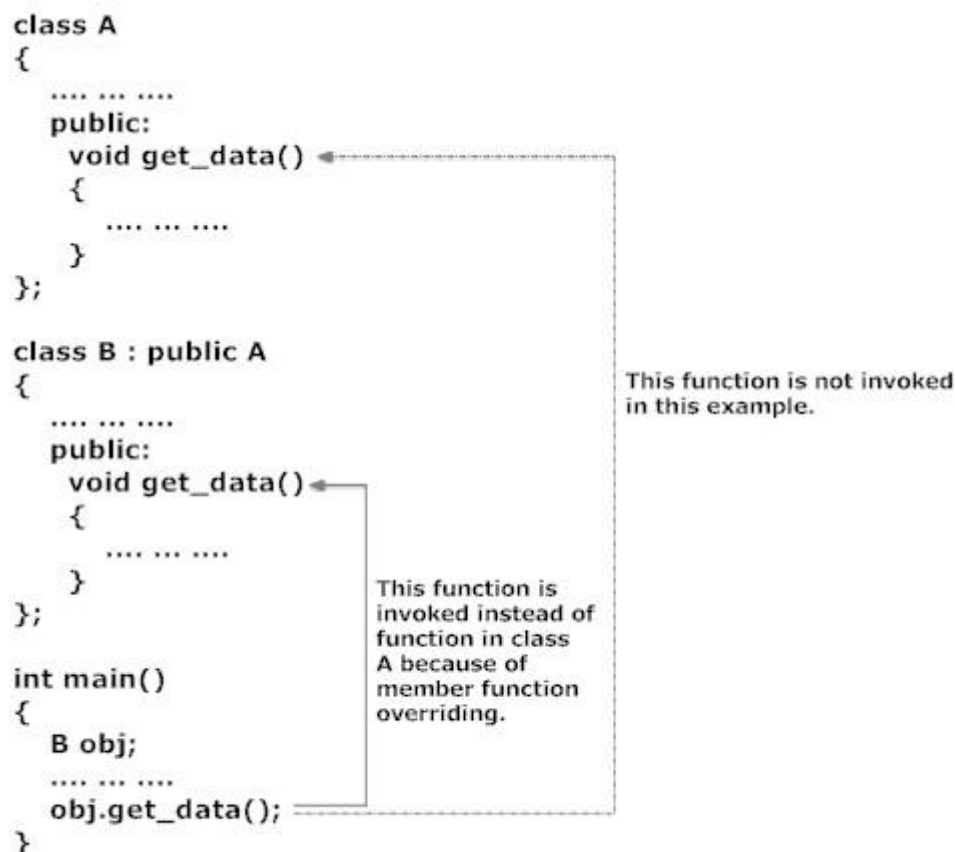


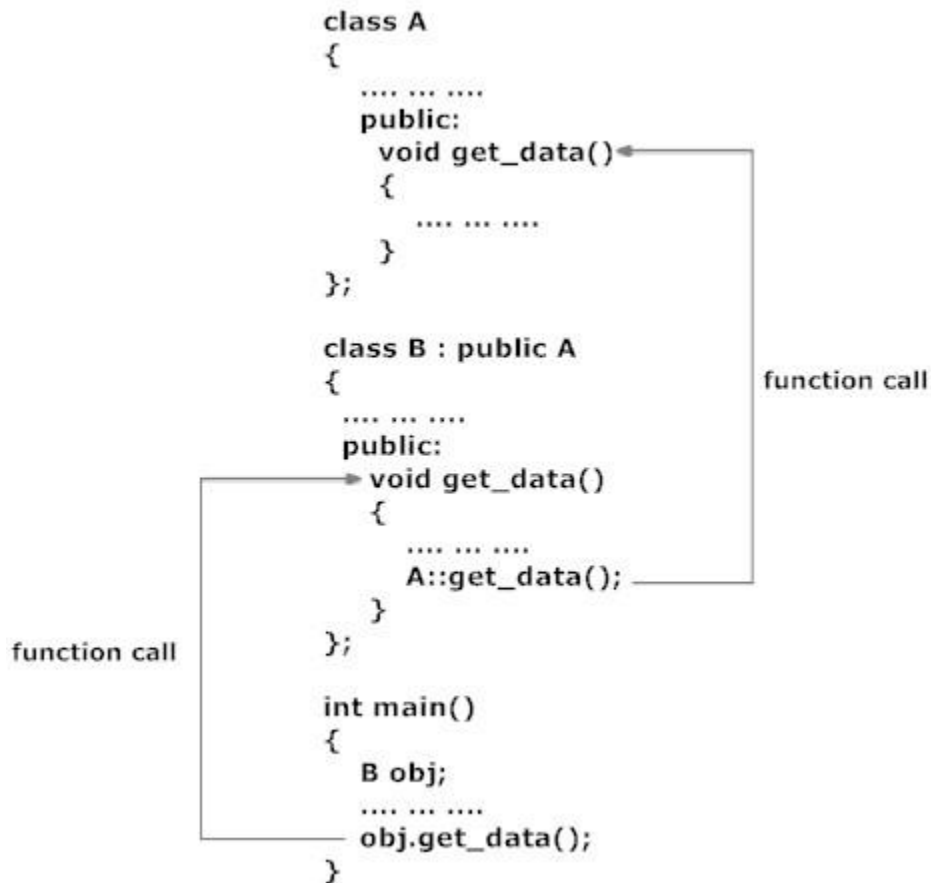
Figure: Member Function Overriding in C++

Accessing the Overridden Function in Base Class From Derived Class

To access the overridden function of base class from derived class, scope resolution operator `::`. For example: If you want to access `get_data()` function of base class from derived class in above example then, the following statement is used in derived class.

```
A::get_data; // Calling get_data() of class A.
```

It is because, if the name of class is not specified, the compiler thinks `get_data()` function is calling itself.



Abstract Class

Abstract Class is a class which contains atleast one Pure Virtual function in it. Abstract classes are used to provide an Interface for its sub classes. Classes inheriting an Abstract Class must provide definition to the pure virtual function, otherwise they will also become abstract class.

Characteristics of Abstract Class

1. Abstract class cannot be instantiated, but pointers and references of Abstract class type can be created.
2. Abstract class can have normal functions and variables along with a pure virtual function.
3. Abstract classes are mainly used for Upcasting, so that its derived classes can use its interface.
4. Classes inheriting an Abstract Class must implement all pure virtual functions, or else they will become Abstract too.

Pure Virtual Functions

Pure virtual Functions are virtual functions with no definition. They start with **virtual** keyword and ends with = 0. Here is the syntax for a pure virtual function,

```
virtual void f() = 0;
```

Example of Abstract Class

```
class Base    //Abstract base class
```

```

{
public:
virtual void show() = 0;      //Pure Virtual Function
};

class Derived:public Base
{
public:
void show()
{ cout << "Implementation of Virtual Function in Derived class"; }
};

int main()
{
Base obj;    //Compile Time Error
Base *b;
Derived d;
b = &d;
b->show();
}

```

Output : Implementation of Virtual Function in Derived class

In the above example Base class is abstract, with pure virtual **show()** function, hence we cannot create object of base class.

Why can't we create Object of Abstract Class ?

When we create a pure virtual function in Abstract class, we reserve a slot for a function in the VTABLE(studied in last topic), but doesn't put any address in that slot. Hence the VTABLE will be incomplete.

As the VTABLE for Abstract class is incomplete, hence the compiler will not let the creation of object for such class and will display an error message whenever you try to do so.

LECTURE-34

Exception Handling:

Exception refers to unexpected condition in a program. The unusual conditions could be faults, causing an error which in turn causes the program to fail. The error handling mechanism of c++ is generally referred to as exception handling.

Generally , exceptions are classified into synchronous and asynchronous exceptions.. The exceptions which occur during the program execution, due to some fault in the input data or technique that is not suitable to handle the current class of data. with in a program is known as synchronous exception.

Example:

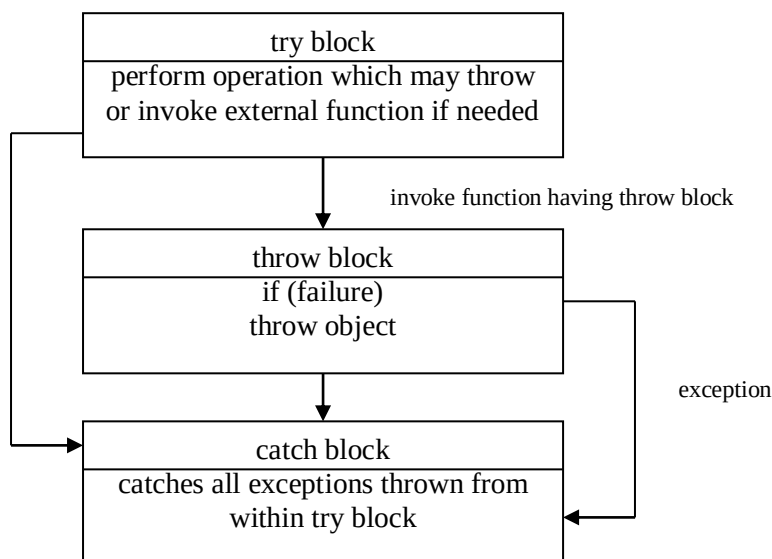
errors such as out of range,overflow,underflow and so on.

The exceptions caused by events or faults unrelated to the program and beyond the control of program are asynchronous exceptions.

For example, errors such as keyboard interrupts, hardware malfunctions, disk failure and so on.

exception handling model:

When a program encounters an abnormal situation for which it is not designed, the user may transfer control to some other part of the program that is designed to deal with the problem. This is done by throwing an exception. The exception handling mechanism uses three blocks: try, throw and catch. The try block must be followed immediately by a handler, which is a catch block. If an exception is thrown in the try block the program control is transferred to the appropriate exception handler. The program should attempt to catch any exception that is thrown by any function. The relationship of these three exceptions handling constructs called the exception handling model is shown in figure:



throw construct:

The keyword throw is used to raise an exception when an error is generated in the computation. the throw expression initialize a temporary object of the type T used in throw (T arg).

syntax:

```
throw T;
```

catch construct:

The exception handler is indicated by the catch keyword. It must be used immediately after the statements marked by the try keyword. The catch handler can also occur immediately after another catch. Each handler will only evaluate an exception that matches.

syn:

```
catch(T)
{
// error messages
}
```

try construct:

The try keyword defines a boundary within which an exception can occur. A block of code in which an exception can occur must be prefixed by the keyword try. Following the try keyword is a block of code enclosed by braces. This indicates that the prepared to test for the existence of exceptions. If an exception occurs, the program flow is interrupted.

```
try
{
...
if (failure)
    throw T;
}
catch(T)
{
...
}
```

example:

```
#include<iostream.h>
void main()
{
int a,b;
cout<<"enter two numbers:";
cin>>a>>b;
try
{
if (b==0)
    throw b;
else
    cout<<a/b;
}
catch(int x)
{
cout<<"2nd operand can't be 0";
}
}
```

LECTURE-35

Array reference out of bound:

```
#define max 5
class array
{
private:
    int a[max];
public:
    int &operator[](int i)
    {
        if (i<0 || i>=max)
            throw i;

        else
            return a[i];
    }
};

void main()
{
    array x;
    try
    {
        cout<<"trying to refer a[1]..."
        x[1]=3;
        cout<<"trying to refer a[13]..."
        x[13]=5;
    }
    catch(int i)
    {
        cout<<"out of range in array references...";
    }
}
```

multiple catches in a program

```
void test(int x)
{
    try{
        if (x==1)
            throw x;
        else if (x==-1)
            throw 3.4;
        else if (x==0)
            throw 's';
    }
    catch (int i)
    {
        cout<<"caught an integer...";
    }
    catch (float s)
    {
        cout<<"caught a float...";
    }
}
```

```
}
catch (char c)
{
cout<<"caught a character...";
}}
void main()
{
test(1);
test(-1);
test(0);
}
```

catch all

```
void test(int x)
{
try{
if (x==1)
    throw x;
else if (x==-1)
    throw 3.4;
else if (x==0)
    throw 's';
}
catch (...)
{
cout<<"caught an error...";
}
```

Module-03:
LECTURE-36

Containership in C++

When a class contains objects of another class or its members, this kind of relationship is called containership or nesting and the class which contains objects of another class as its members is called as container class.

Syntax for the declaration of another class is:

```
Class class_name1
```

```
{
```

```
_____
```

```
_____
```

```
};
```

```
Class class_name2
```

```
{
```

```
_____
```

```
_____
```

```
};
```

```
Class class_name3
```

```
{
```

```
Class_name1 obj1;           // object of class_name1
```

```
Class_name2 obj2;           // object of class_name2
```

```
_____
```

```
_____
```

```
};
```

```

//Sample Program to demonstrate Containership
#include < iostream.h >
#include < conio.h >
#include < iomanip.h >
#include< stdio.h >
const int len=80;
class employee
{
private:
char name[len];
int number;
public:
void get_data()
{
cout << "\n Enter employee name: ";
cin >> name;
cout << "\n Enter employee number: ";
cin >> number;
}
void put_data()
{
cout << " \n\n Employee name: " << name;
cout << " \n\n Employee number: " << number;
}
};
class manager
{
private:
char dept[len];
int numemp;
employee emp;
public:
void get_data()
{
emp.get_data();
cout << " \n Enter department: ";
cin >> dept;
cout << "\n Enter number of employees: ";
cin >> numemp;
}
void put_data()
{
emp.put_data();
cout << " \n\n Department: " << dept;
cout << " \n\n Number of employees: " << numemp;
}
};
class scientist
{
private:
int pubs,year;
employee emp;
public:

```

```

void get_data()
{
    emp.get_data();
    cout << " \n Number of publications: ";
    cin >> pubs;
    cout << " \n Year of publication: ";
    cin >> year;
}
void put_data()
{
    emp.put_data();
    cout << "\n\n Number of publications: " << pubs;
    cout << "\n\n Year of publication: " << year;
}
};
void main()
{
    manager m1;
    scientist s1;
    int ch;
    clrscr();
    do
    {
        cout << "\n 1.manager\n 2.scientist\n";
        cout << "\n  Enter your choice: ";
        cin >> ch;
        switch(ch)
        {
            case 1:
                cout << "\n Manager data:\n";
                m1.get_data();
                cout << "\n Manager data:\n";
                m1.put_data();
                break;
            case 2:cout << " \n Scientist data:\n";
                s1.get_data();
                cout << " \n Scientist data:\n";
                s1.put_data();
                break;
        }
        cout << "\n\n To continue Press 1 -> ";
        cin >> ch;
    }
    while(ch==1);
    getch();
}

```

Difference between Inheritance and Containership :

Containership: Containership is the phenomenon of using one or more classes within the definition of other class. When a class contains the definition of some other classes, it is referred to as composition, containment or aggregation. The data member of a new class is an object of some other class. Thus the other class is said to be composed of other classes and hence referred to as containership. Composition is often referred to as a “has-a” relationship because the objects of the composite class have objects of the composed class as members.

Inheritance: Inheritance is the phenomenon of deriving a new class from an old one. Inheritance supports code reusability. Additional features can be added to a class by deriving a class from it and then by adding new features to it. Class once written or tested need not be rewritten or redefined. Inheritance is also referred to as specialization or derivation, as one class is inherited or derived from the other. It is also termed as “is-a” relationship because every object of the class being defined is also an object of the inherited class.

LECTURE-37

Template:

Template supports generic programming, which allows developing reusable software components such as functions, classes, etc supporting different data types in a single frame work.

A template in c++ allows the construction of a family of template functions and classes to perform the same operation o different data types. The templates declared for functions are called class templates. They perform appropriate operations depending on the data type of the parameters passed to them.

Function Templates:

A function template specifies how an individual function can be constructed.

```
template <class T>
return type functionnm(T arg1,T arg2)
{
fn body;
}
```

For example:

Input two number and swap their values

```
template <class T>
void swap (T &x,T & y)
{
T z;
z=x;
x=y;
y=z;
}
void main( )
{
char ch1,ch2;
cout<<"enter two characters:";
cin>>ch1>>ch2;
swap(ch1,ch2);
cout<<ch1<<ch2;
int a,b;
cout<<"enter a,b:";
cin>>a>>b;
swap(a,b);
cout<<a<<b;
float p,q;
cout<<"enter p,q:";
cin>>p>>q;
swap(p,q);
cout<<p<<q;
}
```

example 2:

find maxium between two data items.

```
template <class T>
T max(T a,T b)
```

```

{
if (a>b)
return a;
else
return b;
}
void main()
{
char ch1,ch2;
cout<<"enter two characters:";
cin>>ch1>>ch2;
cout<<max(ch1,ch2);
int a,b;
cout<<"enter a,b:";
cin>>a>>b;
cout<<max(a,b);
float p,q;
cout<<"enter p,q:";
cin>>p>>q;
cout<<max(p,q);
}

```

Overloading of function template

```

#include<iostream.h>
template <class T>
void print( T a)
{
    cout<<a;
}
template <class T>
void print( T a, int n)
{
    int i;
    for (i=0;i<n;i++)
        cout<<a;
}
void main()
{
    print(1);
    print(3.4);
    print(455,3);
    print("hello",3);
}

```

Multiple arguments function template:

find sum of two different numbers

```

template <class T,class U>
T sum(T a,U b)
{
    return a+(U)b;
}
void main( )

```

```
{  
cout<<sum(4,5.5);  
cout<sum(5.4,3);  
}
```

LECTURE-38

Class Template

similar to functions, classes can also be declared to operate on different data types. Such classes are class templates. a class template specifies how individual classes can be constructed similar to normal class definition. These classes model a generic class which support similar operations for different data types.

syn:

```
template <class T>
class classnm
{
T member1;
T member2;
...
...
public:
T fun();
...
..
};
```

objects for class template is created like:

```
classnm <datatype> obj;
obj.memberfun();
```

example:

Input n numbers into an array and print the element is ascending order.(array sorting)

```
template <class T>
class array
{
T *a;
int n;
public:
void getdata()
{
int i;
cout<<"enter how many no:";
cin>>n;
a=new T[n];
for (i=0;i<n;i++)
{
cout<<"enter a number:";
cin>>a[i];
}
}
void putdata()
{
```

```

for (i=0;i<n;i++)
{
cout<<a[i]<<endl;
}
}
void sort( )
{
T k;
int i,j;
for(i=0;i<n-1;i++)
{
for (j=0;j<n;j++)
{
if (a[i]>a[j])
{
k=a[i];
a[i]=a[j];
a[j]=k;
}
}
}
}
};
void main()
{
array <int>x;
x.getdata();
x.sort();
x.putdata();

array <float> y;
y.getdata();
y.sort();
y.putdata();
}

```

LECTURE-39

Virtual destructors:

Just like declaring member functions as virtual, destructors can be declared as virtual, whereas constructors can not be virtual. Virtual Destructors are controlled in the same way as virtual functions. When a derived object pointed to by the base class pointer is deleted, destructor of the derived class as well as destructor of all its base classes are invoked. If destructor is made as non virtual destructor in the base class, only the base class's destructor is invoked when the object is deleted.

```
#include<iostream.h>
#include<string.h>
class father
{
protected:
char *fname;
public:
father(char *name)
{
fname=new char(strlen(name)+1);
strcpy(fname,name);
}
virtual ~father()
{
delete fname;
cout<<"~father is invoked...";
}

virtual void show()
{
cout<<"father name..."<<fname;
}
};

class son: public father
{
protected:
char *s_name;
public:
son(char *fname,char *sname):father(fname)
{
sname=new char[strlen(sname)+1];
strcpy(s_name,sname);
}
~son()
{
delete s_name;
cout<<"~son() is invoked"<<endl;
}
void show()
{
cout<<"father's name"<<fname;
cout<<"son's name:"<<s_name;
```

```

}
};
void main()
{
father *basep;
basep =new father ("mona");
cout<<"basep points to base object..."
basep->show();
delete basep;
basep=new son("sona","mona");
cout<<"base points to derived object...";
basep->show();
delete basep;
}

```

Overloading of >> and << operator

```

#define size 5
class vector
{
int v[size];
public:
vector();
friend vector operator*(int a,vector b);
friend vector operator *(vector b,int a);
friend istream &operator>>(istream &,vector &);
friend ostream &operator<<(ostream &,vector &);
};
vector :: vector()
{
for(int i=0;i<size;i++)
v[i]=0;
}
vector::vector(int *x)
{
for (int i=0;i<size;i++)
v[i]=x[i];
}
vector operator*(int a,vector b)
{
vector c;
for(int i=0;i<size;i++)
c.v[i]=a*b.v[i];
return c;
}

vector operator*(vector b,int a)
{
vector c;

```

```

for(int i=0;i<size;i++)
c.v[i]=a*b.v[i];
return c;
}
istream &operator>>(istream &din,vector &b)
{
for(int i=0;i<size;i++)
din>>b.v[i];
}
ostream &operator<<(ostream &dout,vector &b)
{
for(i=0;i<size;i++)
dout<<a[i];
return dout;
}
int x[size]={2,4,6};
int main()
{
vector m;
vector n=x;
cout<<"enter elements of vector m";
cin>>m;
cout<<m;
vector p,q;
p=2*m;
q=n*2;
cout<<p;
cout<<q;

}

```